PROJECTS'S DOCUMENTATION GROUP 38

Core System Modules

ptl_scheduler.c

This file implements the core scheduling logic of the Periodic Task Layer (PTL). It manages the high-priority Supervisor task, which is responsible for enforcing strictly periodic task releases, checking deadlines, and applying the appropriate overrun policies (SKIP, CATCH_UP, or KILL) when tasks exceed their allotted execution time.

Functions:

`PTL_Scheduler_Start()`: Creates the high-priority Supervisor task and starts the FreeRTOS scheduler, effectively handing over CPU control to the RTOS.

`prvPTL_SupervisorTask()`: The main control loop of the PTL system. Waking up periodically, it evaluates the state of all registered tasks, releases ready tasks, detects overruns and deadline misses, and safely applies the designated overrun policy using critical sections.

`prvApplyPolicy_Kill()`: A helper function that handles the "KILL" overrun policy. It safely deletes a task that has exceeded its period, resets its internal state, and immediately re-creates a fresh instance of it using its original configuration parameters.

ptl_wrapper.c

This module handles the initialization of the PTL environment and encapsulates user tasks. It maintains the internal task pool, stores global configurations, and provides a generic wrapper function that automatically tracks execution times, deadline misses, and performance statistics for any user-defined job.

Functions:

- `PTL_Init()`: Validates system parameters, initializes the tracing module, and populates the internal task pool by creating FreeRTOS tasks injected with the generic wrapper.
- `PTL_Start()`: Verifies that the PTL system has been successfully initialized and triggers the scheduler startup sequence.
- `PTL_GenericWrapper()`: The universal entry point for all PTL tasks. It suspends the task until released by the Supervisor, executes the user's workload, and upon completion, checks for deadline misses and logs system events.
- `PTL_GetEffectivePolicy()`: Determines the specific overrun policy to apply to a given task, prioritizing the per-task configuration and falling back to the global policy if needed.

- `PTL_GetTaskStats()`: Safely retrieves execution statistics (completed jobs, misses, and overruns) for a specific task using critical sections.
- `PTL_IsTracingEnabled()`, `PTL_GetPolicy()`, `PTL_GetTaskList()`: Utility getters to safely access internal system configurations and the task pool array.
- `prvUint32ToString()`: A lightweight, internal utility used to convert 32-bit integers to strings, completely avoiding the heavy overhead of the standard `printf` library during initialization.

ptl_trace.c

This file implements a low-overhead, tick-precision event tracing and profiling system. It utilizes a circular buffer in RAM to log scheduling events and context switches, allowing for detailed offline analysis of task execution timelines and precise calculation of overall CPU utilization.

Functions:

`PTL_TraceInit()`: Resets the circular trace buffer, its write index, and the accumulated idle time counters, ensuring a clean state before the scheduler starts.

`PTL_LogEvent()`: Safely records a system event (e.g., RELEASE, START, COMPLETE) and its timestamp into the circular buffer. It is designed to be ISR-safe.

`PTL_GetTraceStatistics()`: Aggregates the raw trace data to compute total task releases, completions, deadline misses, overrun events, and calculates the total active CPU utilization percentage.

`PTL_PrintTrace()`: Iterates through the circular buffer and dumps the chronological sequence of recorded trace events to the UART console.

`PTL_PrintStatistics()`: Formats and prints a comprehensive statistical report to the UART console, including a pass/fail check verifying that the PTL system overhead remains below the 10% requirement.

`PTL_TrackIdleEntry()` / `PTL_TrackIdleExit()`: Called by the FreeRTOS context switch hooks to record the exact timestamps when the system enters and exits the Idle task, used to compute CPU load.

`vApplicationIdleHook()`, `vApplicationStackOverflowHook()`, `vApplicationMallocFailedHook()`: FreeRTOS system callbacks to handle idle loops and critical hardware/memory faults.

Utilities & Hardware Drivers

burner.c

A CPU Burner utility designed to simulate precise task workloads. It dynamically calibrates itself to the underlying hardware speed, allowing developers to request a specific amount of CPU time (in milliseconds) for testing purposes.

Functions:

`Burn_Calibrate()`: Runs a benchmark loop measured by the FreeRTOS tick count to determine the CPU speed relative to the loop overhead, calculating the number of iterations required per millisecond.

`Burn()`: Executes a busy-wait loop for the requested number of milliseconds, consuming CPU cycles to simulate a real computational workload.

uart.c

Provides low-level hardware drivers for the UART (Universal Asynchronous Receiver-Transmitter) peripheral. It enables basic serial communication for debugging and printing trace logs to the host console on the MPS2-AN385 platform.

Functions:

- `UART_init()`: Configures the UART peripheral's control registers to establish serial communication.
- `UART_printf()`: Transmits a null-terminated string character by character directly to the UART data register via polling.

startup.c

Contains the bare-metal Cortex-M3 vector table, startup initialization code, and default fault handlers. It sets up the fundamental environment required before the C `main()` function can be safely executed.

Functions:

`Reset_Handler()`: The primary entry point executed immediately upon processor reset. It initializes the system and branches directly to `main()`.

`HardFault_Handler()` & `Default_Handler()`: Exception handlers that catch critical system faults and infinite-loop to prevent undefined behavior.

`prvGetRegistersFromStack()`: A low-level debugging helper that extracts CPU registers from the stack frame during a hardware fault to assist in troubleshooting.

Application & Test Suite

main.c

The main demonstration application for the PTL system. It initializes the hardware, defines three distinct periodic tasks with varying workloads and overrun policies (Sensor/Global, ImgProc/KILL, Logger/SKIP), and starts the real-time scheduler

Functions:

`main()`: The application entry point. Initializes UART, loads the task configurations, calls `PTL_Init()`, and starts the system via `PTL_Start()`.

`vJob_Sensor()`: Simulates a lightweight, well-behaved periodic task.

`vJob_ImageProc()`: Simulates a heavy task that intentionally exceeds its deadline to demonstrate the system's `KILL` policy.

`vJob_Logger()`: Simulates a task that runs late but completes, demonstrating the system's `SKIP` policy.

Test Suite Files

`test_edge_minimal_gap.c`: Validates the scheduler's behavior when tasks use almost 100% of their slice, checking for jitter-induced misses.

`test_load_on_trace.c`: A high-frequency stress test designed to flood the trace circular buffer with events to verify system stability under heavy logging load.

`test_mixed_trace.c`: A complex scenario that runs multiple tasks with conflicting overrun policies (KILL, SKIP, NORMAL) simultaneously to ensure policy isolation.

`test_policy_catchup_trace.c`: Explicitly validates the `PTL_POLICY_CATCH_UP` mechanism, verifying that backlogs are correctly queued and executed.

`test_policy_skip_trace.c`: Explicitly validates the `PTL_POLICY_SKIP` mechanism, ensuring that new releases are cleanly skipped when a task is running late.

`test_preemption.c`: Validates FreeRTOS priority-based preemption, ensuring that a high-priority task successfully interrupts a long-running, low-priority task.

`test_production_check.c`: A formal production check that computes the total system overhead, ensuring the PTL framework consumes ≤10% of the active CPU time.

`test_safety_limits.c`: Verifies the robustness of the `PTL_Init()` function by intentionally passing invalid configurations (NULL pointers, zero tasks) to ensure they are safely rejected.

Core Headers & Data Structures

ptl.h

This header defines the public API and core data structures for the Periodic Task Layer (PTL). It serves as the primary interface for user applications to configure and interact with the real-time scheduler.

Data Structures & Enumerations:

`PTL_OverrunPolicy_t`: An enumeration defining the available system responses when a task exceeds its execution time limit (`PTL_POLICY_SKIP`, `PTL_POLICY_KILL`, `PTL_POLICY_CATCH_UP`, and a `USE_GLOBAL` fallback).

`PTL_GlobalConfig_t`: A structure defining system-wide parameters, including the default global overrun policy, a boolean flag to enable/disable tracing, and the maximum number of allowed tasks.

`PTL_TaskConfig_t`: A user-facing structure used to define the static parameters of a periodic task, including its name, period, relative deadline, priority, stack size, entry function pointer, and an optional per-task overrun policy.

`PTL_TaskObj_t`: The internal runtime representation of a task. It encapsulates the static `PTL_TaskConfig_t` alongside dynamic state variables such as FreeRTOS task handles, current/next release timestamps, active execution flags, and dedicated statistical counters for jobs, misses, and specific policy applications.

ptl_events.h

A dedicated header that isolates the definitions of all system events tracked by the PTL monitoring framework.

Data Structures & Enumerations:

`PTL_EventType_t`: A comprehensive enumeration of all possible system and scheduling events (e.g., `PTL_EVENT_RELEASE`, `PTL_EVENT_START`, `PTL_EVENT_COMPLETE`, `PTL_EVENT_DEADLINE_MISS`, `PTL_EVENT_OVERRUN_KILL`, `PTL_EVENT_SWITCH_IN`, `PTL_EVENT_IDLE_START`). This standardization ensures perfect alignment between the logging engine and the report generator.

ptl_trace.h

This header exposes the API and data structures for the PTL Trace and Monitoring System. It defines how raw scheduling data is stored and how aggregate performance metrics are structured.

Data Structures & Enumerations:

`PTL_TraceRecord_t`: A lightweight structure representing a single event entry within the circular trace buffer. It stores a pointer to the task's name, the specific `PTL_EventType_t`, and the exact tick timestamp of the occurrence.

`PTL_TraceStats_t`: A comprehensive statistics structure populated by the trace analysis engine. It holds aggregated system metrics, including total task releases, successful completions, deadline misses, overall overrun events, accumulated idle time, and the calculated floating-point CPU utilization percentage.

FreeRTOSConfig.h

While primarily the standard FreeRTOS configuration file, this header includes custom macros injected by the PTL framework to enable deep kernel tracin

Key Customizations:

Trace Hooks (`traceTASK_SWITCHED_IN`, `traceTASK_SWITCHED_OUT`): Custom macros that hook directly into the FreeRTOS context switch mechanism. They utilize highly optimized string-matching logic (`IS_IDLE_TASK`) to detect idle time and invoke the appropriate `PTL_LogEvent` or `PTL_TrackIdle` functions with minimal overhead.

Tooling & Utilities

framework.py

This Python script acts as an interactive Command Line Interface (CLI) configurator and simulation launcher for the PTL system. It allows developers to dynamically define task sets, performs mathematical schedulability analysis, automatically generates the corresponding C code, and launches the FreeRTOS simulation on QEMU.

Functions & Routines:

`get_user_config()`: Prompts the user via CLI to input the global default overrun policy and the real-time parameters (Period, WCET, Deadline, Priority, and specific Policy) for up to 8 custom tasks.

`perform_analysis()`: Conducts a pre-flight schedulability analysis on the user-defined task set. It calculates the Minor Cycle (GCD) and Major Cycle (LCM/Hyperperiod), evaluates the total CPU utilization, and runs a tick-by-tick simulated Fixed-Priority Preemptive schedule to predict potential deadline misses before the C code is even compiled.

`generate_code()`: Dynamically generates a valid C source file (`main_gen.c`) based on the analyzed configuration. It writes the auto-generated `main()` function, initializes the `PTL_TaskConfig_t` arrays, and creates automated job bodies using the `Burn()` utility to accurately simulate the requested WCET.

`run_simulation()`: Interfaces directly with the project's `Makefile` using Python's `subprocess` module. It automates the compilation of the newly generated `main_gen.c` and launches the QEMU emulator, gracefully handling errors and cleaning up temporary files upon exit.